Jordan Forbes
ECE-GY 9953 Gen AI Based Chip Design
9/27/26

1. Binary_to_bcd
    a. **Specification/Configuration:** “I am trying to create a Verilog model binary_to_bcd_converter for a binary to binary-coded-decimal converter. The template for the LLM is `module binary_to_bcd_converter( input [4:0] binary_input, output [7:0] bcd_output );`
    b. **Testbench:** The testbench checks all 32 inputs against.`{i/10, i%10}` so there are 32 samples.
    c. **Trajectory:** In iteration 0 the five candidates ranked [0.06, 1.0, 0.53, 0.31, 0.31], and then it stopped there. The loop ended there. The only candidate that passed does the “add 3 if ≥ 5” check *before* each shift. The rest of them corrupted the result after the final shift since they shifted first and then added 3. In the earlier runs there was a typo in the port name so non of the candidates passed in iteration 0. The feedback listed the 15 failing inputs, but the model only rewrote comments or restated the same condition (>= 5 became > 4'd4). It never moved the check before the shift.
    d. **Most Important change:** None was needed in the final run
2. Sequence Detector
    a. **Specification/Configuration:** The template is `module sequence_detector(input clk, input reset_n, input [2:0] data, output sequence_found);`
    b. **Testbench:** The testbench applied the full sequence and expects `sequence_found = 1` on the 8th value. It then applies 4 wrong values and expects 0. There are 12 samples, and each is taken at the rising edge
    c. **Trajectory:** Iteration 0 ranked the candidates [0.92, 0.92, −1, 0.67, 0.17]. It failed only the key check. In iteration 11 the model split the design into a combinational next-state block and a registered sequence_found assignment. It still failed at cycle 8. In iteration 2, the model changed the state-0 transition and make the output depend on `next_state`
    d. **Most Important change:** The feedback did drive a real structural change, to next-state logic without an early output. However the final design has only 5 states for an 8 value sequence.
3. Shift Register
    a. **Specification/Configuration:**I am trying to create a Verilog model for a shift register. It must meet the following specifications: - Inputs: Clock, Active-low reset, Data (1 bit), Shift enable - Outputs: Data (8 bits)

   b. **Testbench:** The testbench runs 8 steps with set reset, data, and enable values and checks data_out after each one. The expected values show that new bits enter at the MSB and the register shifts right.
   c. **Trajectory:** Every single one of the candidates in every iteration ranked 0.25. 28 out of the 30 shifted left with the new bit entering at the LSB. The feedback was identical in every iteration and showed mirrored values.
   d. **Remaining Issue:** The model leaves the shift direction open and the testbench expects the opposite if the models default. The feedback makes the bit reversal easy to see but the model could never figure out the correct direction needed for the shift
4. Abro State Machine
   a. **Specification/Configuration:** I am trying to create a Verilog model for an ABRO state machine. It must meet the following specifications: - Inputs: Clock,Active-low reset, A, B - Outputs: O, State
      Other than the main output from ABRO machine, it should output the current state of the machine for use in verification. The states for this state machine should be one-hot encoded. The template is module abro_state_machine(input clk, input rst_n, input A, input B, output O, output [3:0] State);
   b. **Testbench:** The testbench runs 16 checks of O and state. The state follows the current inputs: A gives 0010, B gives 0001, A and B together give 0100, and neither input holds the state. O = A&B
   c. **Trajectory:** The best design stayed at 5/16 in every iteration. The model kept its sequential state machine but it lags by 1 cycle. Across iterations it only adjusted individual transitions
   d. **Remaining Issue:** Feedback only shows mismatches for particular input pairs, and the model treated each one as a local transition bug instead of rethinking the design
5. Dice Roller
   a. **Specification/Configuration:** // I am trying to create a Verilog model for a simulated dice roller. It must meet the following specifications: - Inputs:, Clock, Active-low reset, Die select (2-bits), Roll - Outputs: Rolled number (up to 8-bits). The design should simulate rolling either a 4-sided, 6-sided, 8-sided, or 20-sided die, based on the input die select. It should roll when the roll input goes high and output the random number based on the number of sides of the selected die. The template is module dice_roller(input clk, input rst_n, input [1:0] die_select, input roll, output [7:0] rolled_number);

   b. **Testbench:** The testbench does 100 rolls per die and checks that each result is in 1…N so there are 4000 samples. It also prints how often each face came up.
   c. **Trajectory:** All 5 iteration-0 candidates scored 0 then the loop stopped
   d. **Most Important change:** This works in simulation only and cannot be synthesized into hardware.
6. LFSR
   a. **Specification/Configuration:** I am trying to create a Verilog model for an LFSR. It must meet the following specifications: - Inputs: Clock, Active-low reset - Outputs: Data (8-bits) The initial state should be 10001010, and the taps should be at locations 1, 4, 6, and 7. The template is `module lfsr(input clk, input reset_n, output [7:0] data);`
   b. **Testbench:** The testbench runs 1 initial state check plus 256 cycle checks against a reference that shifts left.
   c. **Trajectory:** The feedback gave the actual and expected state for each cycle. The model found the right tap set over 2 iterations.
   d. **Most Important change:** The rank went down in iteration 2 even though two of the four taps were now correct.
7. Sequence Generator
   a. **Specification/Configuration:** I am trying to create a Verilog model for a sequence generator. It must meet the following specifications: - Inputs: Clock, Active-low reset, Enable - Outputs: Data (8 bits). While enabled, it should generate an output sequence of the following hexadecimal values and then repeat: 0xAF, 0xBC, 0xE2, 0x78, 0xFF, 0xE2, 0x0B, 0x8D
   b. **Testbench:** The testbench checks 8 values, then 8 more for the repeat, then 1 check that the output holds when enable is low for 17 samples.
   c. **Trajectory:** Iteration 0 ranked [0, −1, −1, 0.06, 0]. Two of the candidates had syntax errors which is why they are -1. For Iterations 1 - 5, every error shows a one-cycle lag. The model's only change that helped was loading 8’hAF on reset, which fixed position 0.
   d. **Remaining Issue:** This is a latency and timing issue. The pattern in the feedback where each value is the previous expected one shows the lag.
8. Traffic Light
   a. **Specification/Configuration:** I am trying to create a Verilog model for a traffic light state machine. It must meet the following specifications: - Inputs: Clock, Active-low reset, Enable - Outputs: Red, Yellow, Green The state machine should reset to a red light, change from red to green after 32 clock cycles, change from green to yellow after 20 clock cycles, and then change from yellow to red after 7 clock cycles. The template is

module traffic_light_fsm(input clk, input reset_n, input enable, output red, output yellow, output green);

   b. **Testbench:** The testbench checks the lights at 4 points in time for 4 samples
   c. **Trajectory:** Iteration 0 ranked [0.75, 0.5, 1.0, 0.5, 0.5], and the loop stopped.
   d. **Most Important change:** The selected design used an enum state with a 6-bit counter that compares against 31, 19, and 6.

9. CPU/ 8-bit Processor
   a. **Specification/Configuration:** “Let us make a brand new microprocessor design together. We're severely constrained on space and I/O. We have to fit in 1000 standard cells of an ASIC, so I think we will need to restrict ourselves to an accumulator based 8-bit architecture with no multi-byte instructions.” I dropped the last sentence because the model needed a module to base its design on.
   b. **Testbench:** The testbench loads a 13-instruction program into RAM with 4 bit opcodes. It expects each instruction to take on a fetch cycle and one execute cycle and it makes 15 checks.
   c. **Trajectory:** For iteration 0 the model return a skeleton with registers and a PC that increments. There was no decoder and no ALU. For iterations 1-8 The feedback was the full PASS/FAIL list. The model added an instruction decoder with its own ISA. For iterations 9-10 the model adds @(posedge clk); inside its always block to “wait a cycle” for each instruction. None of the candidates failed

Conclusion

AutoChip generated a design that compiles, used required register and memory names, resets correctly and has a simple ALU. Although it compiles, The ISA does not match the expected encoding, memory is not used for operands and more things are left incomplete. For the CPU example the feedback that worked for the smaller examples doesn’t work here.11 failures all came from 1 missing piece but the PASS/FAIL list doesn't show that